

Lab 07 — Adaptive Maintenance of Codeface

Target: [siemens/codeface](#) @ [e6640c93](#) (last commit 2019-08-22, repository archived 2022)
Analysed project: [pallets/flask](#) — 5 546 commits, 69 release tags **Host:** Ubuntu 24.04.2 LTS, Docker 28.3.3, 12 cores, 15 GiB RAM **Date:** 2026-08-11

1. Outcome

Codeface runs. A full seven-range analysis of Flask completes in **2 minutes 5 seconds with zero errors**, ending in => **Codeface run complete!** (exit code 0).

Success criterion	Status
Core dependencies installed and linked	32/32 required R packages verified <i>loadable</i> ; all Python 2.7 deps import
Backing services running and communicating	MySQL 5.7 with 41 tables, reached independently by Python (MySQLdb) and R (RMySQL); Node.js ID service answering on :8080
Analysis run executed on a target repository	7 release ranges of Flask, 1 223 commits, exit 0
Expected output generated without fatal crashes	1 223 commit rows + 315 developers + 40 clusters in the database; 255 output files including 89 PDFs and 88 sociograms

Independently corroborated by Codeface's **own** test suite: 4/4 integration tests pass, and 28/29 unit tests (§7.2 explains the one expected failure).

Two latent bugs in Codeface itself were found and fixed along the way (§5.3, §5.4).

2. Why the project does not run today

Codeface's documented installation path is `vagrant up` against an **Ubuntu 14.04 (Trusty)** box, provisioned by `integration-scripts/*.sh`. That path is not merely inconvenient today — it is impossible:

- `install_repositories.sh` hard-aborts on anything that is not 14.04 or 16.04: `*) echo "Unsupported version of Ubuntu detected, aborting"`.
- Both of those releases are past end-of-life, and I confirmed their package archives no longer resolve (`old-releases.ubuntu.com/.../xenial` → **404**).
- The stack it wants — Python 2.7 (EOL 2020), R 3.x, MySQL **5.5** (Travis explicitly *downgrades* 5.6 → 5.5) — has no counterpart on a 2026 host, which ships Python 3.12 and no R or MySQL at all.

So the work is not "install the dependencies". It is: decide which era to reconstruct, find which pieces of that era still exist, and rebuild the rest.

3. Strategy

Reconstruct the legacy runtime inside a container pinned to the newest distribution that still satisfies Codeface's constraints, rather than trying to force 2013-era code onto a 2026 host. The Codeface checkout and the analysis data are **bind-mounted**, not baked into the image, so source patches take effect immediately without a rebuild — which matters when the debugging loop is the bulk of the work.

3.1 Choosing the base — an availability probe

Before writing anything I checked what still exists:

Source	Result
<code>old-releases.ubuntu.com/ubuntu/dists/xenial</code>	404 — 16.04 archives gone
<code>archive.ubuntu.com/ubuntu/dists/bionic</code>	200 — 18.04 still live
<code>cloud.r-project.org/bin/linux/ubuntu/bionic-cran35</code>	200 — R up to 3.6.3

Source	Result
<code>packagemanager.posit.co/cran/_linux__bionic/<date></code>	200 — date-pinned CRAN
<code>bootstrap.pypa.io/pip/2.7/get-pip.py</code>	200
PyPI MySQL-python, python-ctags, progressbar	200

Ubuntu 18.04 (bionic) is the answer: the newest Ubuntu that still ships `python2.7` in main *and* still has live archives, and for which CRAN still hosts an R build.

A detail that turned out to matter a lot: `codeface/util.py:check4ctags()` asserts the ctags version string *starts with* `Exuberant Ctags 5.9~svn20110310` and raises otherwise. Bionic ships exactly `1:5.9~svn20110310-11ubuntu0.1`. On any modern distro — which ships *universal*-ctags — this check can never pass, and `proximity` tagging is unusable without patching Codeface.

3.2 The pinned stack

Component	Version	Why this one
Ubuntu	18.04.6	last with <code>python2.7</code> in main + live archives
Python	2.7.17	what Codeface is written in
R	3.6.3	newest in CRAN's frozen <code>bionic-cran35</code> ; matches Codeface's 2019 era
CRAN	snapshot 2020-04-01	deliberately dated <i>just before</i> R 4.0.0 shipped, so every resolved package still declares R 3.x compatibility
Bioconductor	3.10	the release matched to R 3.6
MySQL	5.7.42	bionic's default, with 5.5 semantics emulated (§4.3)

Component	Version	Why this one
Node.js	14.21.3	newest line still running on bionic's glibc 2.27

4. Chronological narrative

4.1 Scoping the R dependency set

Upstream `packages.r` installs ~45 packages and is unusable as written, for four independent reasons:

1. It calls `biocLite()` from `bioconductor.org/biocLite.R`, retired in 2019 (Bioconductor 3.8+); the URL now only emits an error.
2. It installs from **live CRAN HEAD**, whose current versions require $R \geq 4.x$ and refuse to install on R 3.6.
3. It pulls four packages from GitHub via `devtools::install_github(tm.plugin.mail, snatm, shinyGridster, shinybootstrap2)` — all used only by the mailing-list and Shiny front-end paths, not by `codeface run`.
4. It passes `dependencies=TRUE`, i.e. resolves **Suggests** as well.

Rather than install all 45, I computed the transitive `source()` closure of the three R scripts that `codeface run` actually executes — `R/cluster/persons.r`, `R/complexity.r`, `R/analyse_ts.r`. That reaches 23 R files needing **33 libraries** (29 CRAN + 3 Bioconductor + 2 base). The mailing-list-only packages are not on this path at all.

4.2 Three build attempts

Attempt 1 — aborted. Pinned CRAN to the 2020-04-01 snapshot but kept `dependencies=TRUE`. Parsing the snapshot's `PACKAGES` index showed why this was hopeless: with Suggests resolved, the 29 roots expand to a **1 544-package closure**; without them, **96**. It was downloading `usethis` (a devtools dependency) when I killed it.

Considered and rejected. Michael Rutter's `ppa:marutter/c2d4u3.5` is still alive for bionic with 6 201 prebuilt `r-cran-*` debs covering nearly everything needed — very tempting. Rejected because those debs are built against **R 4.4** (e.g. `r-cran-igraph 1.3.5`) while we run R 3.6.3. Beyond the ABI mismatch, R 4.x flips the `stringsAsFactors` default and carries a decade of igraph API drift — exactly the class of silent breakage that would quietly corrupt results from 2013-era code rather than failing loudly.

Attempt 2 — the `libgmp` failure. With dependencies restricted to `c("Depends", "Imports", "LinkingTo")` the build got much further, then:

Error: package or namespace load failed for 'igraph' in dyn.load(file, DLLpath = DLLpath, ...):

unable to load shared object '/usr/local/lib/R/site-library/igraph/libs/igraph.so':

libgmp.so.40: cannot open shared object file: No such file or directory

ERROR: required packages failed to install: igraph, markovchain, BiRewire

`markovchain` and `BiRewire` were collateral damage — both depend on igraph.

The log line `* installing *binary* package 'igraph'` corrected an assumption I had made earlier: **97 of the 101 packages were installed as prebuilt binaries**; only the four Bioconductor ones compiled from source. Posit's `PACKAGES` index advertises source tarballs, but the actual downloads are content-negotiated on the R `User-Agent` and return binaries. That is why the build took 8 minutes rather than hours.

The consequence is subtle and worth stating plainly: `install.packages()` reported **success for igraph**. A prebuilt binary is unpacked, never linked, so a missing shared-library dependency is not detected at install time — it surfaces much later, at `dyn.load()`. I confirmed the exact requirement by downloading the artefact Posit actually serves and reading its ELF headers:

```
$ objdump -p igraph/libs/igraph.so | grep NEEDED
```

```
NEEDED libxml2.so.2  NEEDED libgmp.so.10  NEEDED libgmp.so.40
```

```
NEEDED liblapack.so.3  NEEDED libblas.so.3  NEEDED libgfortran.so.4
```

`libgmp.so.10` and `libgmp.so.40` were absent because I had built a deliberately slim image, whereas Posit's build host was fuller.

Attempt 3 — success. Added an APT layer for `libgmp-dev libgmp-dev libmpfr-dev libicu-dev libblas-dev liblapack-dev`, placed immediately before the R package layer so the expensive earlier layers stayed cached. I also changed the verifier to print the actual `dyn.load` message per failing package instead of just naming it, so the next iteration would not need the same manual forensics.

Result: **All 32 required R packages installed and loadable**, 145 packages in the library.

Design note. The verifier checks that every required package *loads*, not merely that it installed. That distinction is the entire reason the `libglib` problem was caught at build time instead of halfway through an analysis run.

4.3 Python 2.7 and MySQL

pip was bootstrapped from `bootstrap.pypa.io/pip/2.7/get-pip.py` (pip 20.3.4 / setuptools 44.1.1); bionic's own `python-pip` is too old to negotiate with today's PyPI. Every dependency is pinned to its last py2.7-compatible release — unpinned installs resolve to py3-only versions and fail. `MySQL-python==1.2.5` compiled cleanly against bionic's `libmysqlclient-dev`.

`jira` is required even for `codeface run`, which is not obvious: `project.py` imports `codeface.conway`, which imports `jira` at module scope.

MySQL 5.7 standing in for 5.5 needed one real adaptation: `sql_mode` is emptied in `/etc/mysql/conf.d/codeface.cnf`. MySQL 5.7 enables `ONLY_FULL_GROUP_BY`, `STRICT_TRANS_TABLES` and `NO_ZERO_DATE` by default, none of which existed in 5.5, and Codeface's 2013-era SQL does not satisfy them. This is what Travis was really buying by downgrading 5.6 → 5.5.

The three Vagrant-era provisioning scripts were replaced with one idempotent `docker/bootstrap_codeface.sh`; they assume a VM with a `vagrant` user, passwordless `sudo` and `chown vagrant.vagrant`. Because `root@localhost` authentication differs depending on how the package self-initialised, the bootstrap *probes* for a working admin login rather than assuming one.

4.4 Node.js — a failure my own Dockerfile hid

The first bootstrap ended with `npm: command not found`, even though the image had built successfully. The reason was a bug in my own Dockerfile:

```
RUN apt-get install -y --no-install-recommends nodejs npm && ... || true
```

The trailing `|| true` applies to the **whole && chain**, so it swallowed a genuine apt failure that was sitting in the build log all along:

```
npm : Depends: node-gyp (>= 0.10.9) but it is not going to be installed
```

```
E: Unable to correct problems, you have held broken packages.
```

`--no-install-recommends` prevents `node-gyp`'s dependencies from resolving, so bionic's `npm` is uninstallable under that flag. Rather than drop the flag, I installed the **official Node 14.21.3 tarball** — the newest line that still runs against bionic's glibc 2.27 (Node 16+ needs

2.28), bundling npm 6, which negotiates with the current registry far better than bionic's npm 3.5.2. `nodejs` is symlinked to `node` because Codeface invokes the Debian name.

Lesson recorded: never terminate a Dockerfile `RUN` chain with `|| true` — it converts a build failure into a runtime failure far from its cause.

5. The analysis run, and the four defects it exposed

I ran a **single-range smoke configuration** (`conf/flask-smoke.conf`, 3.0.0 → 3.1.0) first so each debug cycle cost minutes rather than an hour.

5.1 `svglite` — a dependency no static analysis can find

The first smoke run reached the *final* stage and died:

```
[codeface.R] CRITICAL: there is no package called 'svglite'
```

```
4: ggsave(file.path(graphdir, "ts_commits.svg"), g, width = 12, height = 8)
```

```
6: svglite::svglite
```

`svglite` appears **nowhere** in Codeface's source. I re-scanned the entire run-path closure for `library()`, `require()`, `requireNamespace()` and `pkg::` references; the only namespace-qualified package is `igraph`. The dependency is indirect: `analyse_ts.r` calls `ggsave()` with a `.svg` filename, and `ggplot2` dispatches on the extension to `svglite::svglite` at runtime. `ggplot2` declares `svglite` only under **Suggests** — precisely the field my dependency restriction dropped.

So the 16× build-time saving had exactly one casualty, and it was invisible to static analysis. The general lesson: a source-derived dependency closure is a **lower bound**; runtime dispatch escapes it.

5.2 CLI option levels (my bug, not Codeface's)

My wrapper invoked `codeface --recreate run ...` and `argparse` rejected it. `-j/-l/-f` are *global* options that must precede the `run` subcommand, while `--recreate/--no-report/--profile-r` belong to `run` itself.

With that fixed, the smoke run finished: => **Codeface run complete!**, exit 0.

5.3 Upstream bug — project configs cannot enable `sloccount`

The run succeeded, but `sloccount_ts` was empty. Re-running the complexity stage at `--loglevel devinfo` showed `sloccount : FALSE` — even though `conf/flask-smoke.conf` says `sloccount: true`. The cause is in `codeface/R/config.r::load.config`:

```
if(is.null(conf$understand)) { conf$understand <- FALSE }
```

```
if(is.null(conf$sloccount)) { conf$sloccount <- FALSE }
```

```
...
```

```
conf <- c(conf, yaml.load_file(project.file)) # appends, does not merge
```

The defaults are applied to the global config **before** the project file is merged, and `c()` *concatenates* lists rather than merging them. The result holds **two** elements named `sloccount`, and `conf$sloccount` returns the first — the default `FALSE`. So `sloccount: true` in *any* project config was silently discarded and `do.complexity.analysis()` always returned early.

The Python side does not share this bug: `configuration.py` uses `dict.update()`, which replaces correctly. Only the R loader is affected, and only for the two keys defaulted before the merge.

Fix: replace duplicate keys instead of appending.

```
project.conf <- yaml.load_file(project.file)
```

```
conf[names(project.conf)] <- project.conf
```

Result: `sloccount : TRUE`, and the stage reports `Analysing 41 code samples`.

5.4 Upstream bug — `sloccount` races itself under `-j > 1`

With `sloccount` finally enabled, the log showed `running command '/usr/bin/sloccount ...' had status 1`, and only **6 of 41** samples reached the database.

`sloccount` alone worked fine (10 075 SLOC for Flask 3.1.0, exit 0), and so did two sequential runs. Running four *concurrently* reproduced it instantly:

```
mkdir: cannot create directory '/root/.slocdata': File exists
```


run1 EXIT=1 run2 EXIT=1 run3 EXIT=1 run4 EXIT=1

`gather.sloccount.results()` invokes `sloccount` without `--datadir`, so every instance uses the single default `~/.slocdata`. `do.complexity.analysis()` calls it from inside `mcldapply.db()`, so with `-j > 1` the instances race and all but one abort. The failures were invisible because `do.system` redirects `stderr` to `/dev/null`; the only symptom was a sparsely populated table.

Fix: give each invocation its own data directory. One wrinkle found by testing — `sloccount` requires the directory to *already exist*, and fails with `Error: <path> is not a directory` otherwise — so the fix uses `tempfile() + dir.create() + on.exit(unlink(...))`.

Verified: four concurrent runs all exit 0, and re-running the complexity stage with `-j 4` wrote **41 of 41** samples.

6. Patches applied

All on branch `lab07-modernisation`.

File	Change
<code>setup.py</code>	Removed the bogus 'VCS' dependency (§6.1)
<code>codeface/R/config.r</code>	Fixed project-config merge so <code>sloccount/understand</code> are honoured (§5.3)
<code>codeface/R/sloccount.r</code>	Per-invocation <code>--datadir</code> , making the complexity stage concurrency-safe (§5.4)
<code>conf/flask.conf</code>	New — 7-range Flask analysis configuration
<code>conf/flask-smoke.conf</code>	New — single-range configuration for fast debugging

6.1 The VCS dependency

`setup.py` declared `install_requires=[..., 'VCS', ...]`. Codeface's version-control abstraction is the **in-tree** module `codeface/VCS.py`. `VCS` on PyPI is an unrelated 2010–2011 package ("Various version Control System management abstraction layer" by a different author),

which I verified via the PyPI API. Declaring it makes pip download foreign code that is never imported.

7. Evidence

7.1 Database population

Full Flask run, `codeface` database:

Release ranges analysed

```
+-----+-----+-----+-----+
| range_id | start_tag | end_tag | commits |
+-----+-----+-----+-----+
|    1 | 1.0     | 1.1.0  |   257 |
|    2 | 1.1.0   | 2.0.0  |   312 |
|    3 | 2.0.0   | 2.1.0  |   201 |
|    4 | 2.1.0   | 2.2.0  |   118 |
|    5 | 2.2.0   | 2.3.0  |   127 |
|    6 | 2.3.0   | 3.0.0  |    85 |
|    7 | 3.0.0   | 3.1.0  |   123 |
+-----+-----+-----+-----+
```

Table	Rows
<code>commit_dependency</code>	2 942
<code>timeseries</code>	1 824
<code>commit</code>	1 223

Table	Rows
edgelist	808
pagerank_matrix	724
cluster_user_mapping	395
author_commit_stats	362
person	315
sloccount_ts	234
cluster	40
release_timeline	15
pagerank	14
plots	10
release_range	7

Cross-check: the per-range commit counts were verified against `git rev-list --no-merges --count` run directly on the Flask repository — they match exactly, range for range ($257 + 312 + 201 + 118 + 127 + 85 + 123 = 1\,223$). The results are not merely present, they are *correct*.

The results are also plausible on their face: the top committer is **David Lord** (552 commits), Flask's lead maintainer, and **Armin Ronacher**, Flask's original author, appears in the developer list.

7.2 Codeface's own test suite

Suite	Result
Integration (<code>test_exampleprojects</code>)	4/4 passed in 81 s — ===== All integration and unit tests succeeded :) =====
Unit	28/29 passed

The single unit failure is `test_cppstats_works`. `cppstats` is only needed for `feature/feature_file` tagging, not the `proximity` tagging used here — and it is **no**

longer installable at all: `install_cppstats.sh` fetches srcML from <http://131.123.42.38/lmcrrs/beta/srcML-Ubuntu14.04-64.deb>, a host that now times out. That analysis mode is permanently unreachable via the documented path.

`per_cluster_statistics` remains empty. This is *expected upstream behaviour*, not a defect of this setup: Codeface's own integration test lists that table under `ignore_tables` with the comment `# TODO: Check if these should be filled`. Every table upstream *does* expect (`cluster`, `commit`, `person`, `plots`, `release_range`, `release_timeline`) is populated.

7.3 Reproducibility check

`svglite` was initially installed into the *running* container to keep the debugging loop moving, which meant the image definition was, briefly, not the thing that had been proven to work. To close that gap the image was rebuilt from the Dockerfile alone and a **fresh container** was created from it:

All 33 required R packages installed and loadable.

Total packages in library: 133

...

=> Codeface run complete!

```
+-----+-----+-----+-----+
| commits | people | clusters | sloccount_rows |
+-----+-----+-----+-----+
|  123 |   24 |    3 |         41 |
+-----+-----+-----+-----+
```

The 41 `sloccount_ts` rows are the load-bearing number: they only appear if both R source patches (§5.3, §5.4) are in effect, and the run only completes at all if `svglite` is baked into the image. The environment is reproducible from its definition, not from accumulated manual state.

7.4 Generated files

255 files under `work/res/flask/proximity/`:

- **7 LaTeX-compiled PDF reports**, one per release range (`report-1.0_1.1.0.pdf` ...)

- **88 sociogram files** (`sg_*.dot` / `sg_*.pdf`) — developer collaboration graphs
- **89 PDFs** total, including time-series and cluster plots
- per-range `adjacencyMatrix.txt`, `top20_*.tex`, `vcs_analysis.db`

A generated sociogram is genuine igraph output with community detection:

```
/* Created by igraph 1.2.5 */
```

```
digraph {
```

```
graph [ label="Spin Glass Community 1 Community Quality = 0.706490384615385" ];
```

```
0 [ name=1 label=pgjones fontsize=50 fillcolor=grey60 style=filled
```

7.5 The web frontend

The frontend serves on <http://localhost:8081/>, driven by the same database the analysis populated.

The documented route is dead. `docs/webserver.md` instructs you to install a custom fork of shiny-server ([JohannesEbke/shiny-server](#), branch `no-su`) into a bundled **Node.js 0.10.13** — a 2013 runtime that no longer builds, from a fork that has since disappeared.

That route is also unnecessary. The `codeface dynamic` subcommand runs the same Shiny apps directly through R's own built-in web server:

```
Rcode = "library(shiny); runApp(host='0.0.0.0', port={})".format(args.port)
```

No Node, no shiny-server. Two packages were still missing:

Package	Upstream source	Status
<code>shinybootstrap2</code>	<code>devtools::install_github("rstudio/shinybootstrap2")</code>	That repo now 404s — but the package <i>is</i> on CRAN, so the pinned snapshot supplies it (0.2.1)
<code>shinyGridster</code>	<code>devtools::install_github("wch/shiny-gridster")</code>	Genuinely GitHub-only; still alive. Installed from a source tarball via R CMD INSTALL

Package	Upstream source	Status
		rather than pulling the whole devtools tree into the image

Every Shiny app wraps its UI in `shinybootstrap2::withBootstrap2()`, so without it *all six* apps fail with `ERROR: there is no package called 'shinybootstrap2'`. `shinyGridster` is needed only by `dashboard`.

Verified from the host:

```
$ curl -s -o /dev/null -w "%{http_code}" http://localhost:8081/
```

200

The served page is real Shiny output (`shiny[1.4.0.2]`, `bootstrap[2.3.2]`), and the app's own data query returns the analysed project:

```
id name analysisMethod
```

```
1 2 flask    proximity
```

The `dashboard` app also returns 200 with its `gridster/justgage` assets.

```
docker exec -d codeface bash /scripts/serve_frontend.sh projects # or dashboard, plots, ...
```

8. Reproducing

```
git clone https://github.com/siemens/codeface.git
```

```
mkdir -p work/git-repos work/res
```

```
git -C work/git-repos clone https://github.com/pallets/flask.git
```

```
./docker/up.sh # build image + bootstrap services
```

```
docker exec -e JOBS=4 codeface \
```

```
    bash /scripts/run_analysis.sh conf/flask.conf # run the analysis
```

```
docker exec codeface bash /scripts/collect_evidence.sh flask
```

```
docker exec -d codeface bash /scripts/serve_frontend.sh projects # → localhost:8081
```

Files added for this lab:

Path	Purpose
<code>docker/Dockerfile</code>	the pinned legacy runtime
<code>docker/install_r_packages.R</code>	replaces upstream <code>packages.r</code>
<code>docker/python_requirements_pinned.txt</code>	last py2.7-compatible releases
<code>docker/bootstrap_codeface.sh</code>	replaces the three Vagrant provisioning scripts
<code>docker/start_services.sh</code>	MySQL + ID service, idempotent
<code>docker/run_analysis.sh</code>	analysis wrapper
<code>docker/collect_evidence.sh</code>	database evidence queries
<code>docker/serve_frontend.sh</code>	web frontend on :8081, bypassing the dead shiny-server route
<code>docker/up.sh</code>	one-command bring-up

9. Limitations

- **Mailing-list analysis (`codeface ml`) was not attempted.** It needs `snatm` and `tm.plugin.mail` from GitHub forks, plus `wordnet`. Flask coordinates on GitHub rather than a mailing list, so there was nothing to analyse; `codeface run` does not touch this path.
- **Feature tagging is permanently unavailable** — srcML's download host is dead (§7.2).
- **The web frontend runs, but not by the documented route** (see §7.5). The `shiny-server` path in `docs/webserver.md` is dead.
- **Conway analysis** requires a JIRA issue tracker and Titan jars; not applicable to Flask. (`hashmap`, which it uses, was the only optional package that failed to install — it was archived from CRAN.)
- The container runs as root, so files it writes into the bind mount are root-owned on the host.